

EXAMINATION ASSIGNMENT

Google Maps Competitor Update Intelligence Tool

Technical Development Evaluation

1. Assignment Objective

Build a complete web-based competitor intelligence tool focused specifically on Google Maps Updates. The system should collect Google Maps update/post data from a manually defined set of competitor business profiles, store the data locally, analyze what competitors are publishing, identify recurring topics and trends, and use the collected repository to generate new Google Maps update ideas and complete draft posts.

The tool should be simple enough for a non-technical user to operate through a web interface, while the underlying system should demonstrate strong technical architecture, scraping, data storage, AI processing, duplicate detection, analytics, and deployment.

2. Project Structure

The system should work project by project.

Example:

- 🕒 Project: ABC Salon
- 🕒 Our Profile: ABC Salon, Kharghar
- 🕒 Competitors: Competitor A, Competitor B, Competitor C
- 🕒 Keywords: salon in Kharghar, salon in Bandra, salon in Khar

Competitors and keywords belong to the project. A competitor does not need to be directly associated with a particular keyword.

3. Core Workflow

1. Create a project.
2. Add the business/profile being researched.
3. Manually add competitor businesses.
4. Optionally add search keywords.
5. Start a scraping job.
6. Open/search the relevant Google Maps business profiles.
7. Extract available Google Maps Updates/Posts.
8. Download/store the post text, images and relevant metadata.
9. Detect and ignore posts that already exist in the repository.
10. Store the newly collected information in the project's repository.
11. Analyze the collected posts using AI.
12. Identify topics, keywords, content patterns and calls to action.
13. Generate trend information across competitors.
14. Generate new Google Maps update ideas or complete draft updates when requested.

4. Scope of Data

The core scraper should focus ONLY on Google Maps Updates/Posts.

For each post, capture as much useful information as is reliably available, including:

- ⌚ Competitor/business name
- ⌚ Google Maps profile URL
- ⌚ Google Maps post/update URL, where available
- ⌚ Post/update text
- ⌚ Published date, where available
- ⌚ Images/media published with the update
- ⌚ Call to action
- ⌚ Detected keywords
- ⌚ Detected industry topic
- ⌚ Scrape date/time
- ⌚ Project association
- ⌚ Source/reference information

5. Competitor Management

Competitors must be added manually in the core version.

- ⌚ Add competitor
- ⌚ Edit competitor
- ⌚ Remove competitor
- ⌚ View competitor's collected updates
- ⌚ Scrape/re-scrape a competitor
- ⌚ View the last scraping status
- ⌚ See how many posts were collected

The same competitor may exist in multiple projects. The system should avoid unnecessary duplicate records where the same source post has already been collected.

6. Keyword-Based Discovery

Keywords are part of the project and can be used as an additional discovery mechanism. The core requirement is that competitors are manually entered.

If a keyword is used to discover businesses, the system should compare discovered businesses with the existing competitor repository before creating duplicates.

7. Scraping Engine

The scraping component should be Python-based. Selenium is the preferred approach for the live Google Maps workflow because the target site is dynamic and may restrict automated access.

- ⌚ Process competitors one by one.
- ⌚ Navigate to the relevant Google Maps profile.
- ⌚ Locate the available Updates/Posts.
- ⌚ Extract the available data.
- ⌚ Download media where technically and legally appropriate.
- ⌚ Store the extracted data in persistent storage.
- ⌚ Record success, failure and error details.
- ⌚ Support re-scraping without unnecessarily duplicating previously collected posts.

8. CAPTCHA and Anti-Automation Handling

Google may present CAPTCHA, verification, rate limits, or other anti-automation controls. The system must handle this situation gracefully.

- ⌚ If a CAPTCHA/verification screen appears, the application should notify the user and provide a way to continue after manual verification.
- ⌚ The scraper should not silently fail when verification is required.
- ⌚ The scraping job should record that manual intervention was required.
- ⌚ Students may research technically appropriate ways to improve scraper reliability, but must respect applicable laws, platform terms, and security controls.

BONUS: If a student demonstrates a reliable way to reduce CAPTCHA interruptions using permitted methods or legitimate third-party CAPTCHA-solving infrastructure, this may receive the special bonus mentioned in the examination instructions.

9. Demo and Pre-Scraped Repository

Because Google Maps scraping can be affected by CAPTCHA and other external factors, students must not depend entirely on a live scrape for evaluation.

- ⌚ The deployed system must contain a working pre-populated demo dataset.
- ⌚ The dataset should contain realistic scraped Google Maps update records.
- ⌚ The evaluator must be able to browse, search, analyze and generate content from the stored data.
- ⌚ Students must also demonstrate the live scraping workflow.
- ⌚ A video should be submitted showing the scraping workflow, including what happens when verification/CAPTCHA is encountered.

10. Data Repository

Create a persistent repository for all collected competitor updates.

The repository should allow the user to:

- ⌚ Search posts
- ⌚ Filter by project
- ⌚ Filter by competitor
- ⌚ Filter by date
- ⌚ Filter by topic
- ⌚ Filter by keyword
- ⌚ View images
- ⌚ View original/source URLs
- ⌚ View scraping history
- ⌚ Identify duplicate/previously collected posts

11. Duplicate Detection

Duplicate handling is a core requirement. When additional scraping is performed, the system should not create duplicate records for posts already present in the repository.

The implementation should use a sensible unique identifier or fingerprint based on available source information, such as the post URL, source ID, or a combination of stable fields.

12. AI-Based Content Analysis

Once sufficient competitor data has been collected, the system should analyze the repository using an AI API.

The analysis should identify information such as:

- ⌚ Main topic
- ⌚ Sub-topic
- ⌚ Keywords
- ⌚ Content type
- ⌚ Call to action
- ⌚ Offer/promotion patterns
- ⌚ Frequently discussed subjects
- ⌚ Content frequency
- ⌚ Competitor-level publishing patterns

13. Trend Analysis

The system should calculate how frequently topics appear across the competitor repository.

Example:

Topic	Competitors Using Topic	Occurrence
Hair Care Tips	25 of 50	50%
Festival Offer	18 of 50	36%
Before/After	12 of 50	24%

The purpose is to help identify subjects that are repeatedly being used in the market and provide data that can support future Google Maps content planning.

14. Content Idea Generator

After building the repository, the user should be able to request a specific number of content ideas.

Examples:

- ⌚ Generate 3 ideas
- ⌚ Generate 10 ideas
- ⌚ Generate 50 ideas

The number requested by the user should control the number of generated ideas.

15. Complete Google Maps Update Generator

The generated content should be more than a title.

Each generated update should ideally include:

- ⌚ Topic
- ⌚ Complete Google Maps update copy
- ⌚ Relevant keywords
- ⌚ Call to action
- ⌚ Suggested image concept
- ⌚ AI-generated image, where supported
- ⌚ Project/client association

The generated content should be informed by the collected repository rather than being generic content unrelated to the competitor research.

16. Duplicate Prevention for Generated Ideas

If the user generates ideas today and requests more ideas later, the system must avoid generating the same previously generated ideas again.

For the core requirement, duplicate prevention should compare against ideas previously generated for the same project/profile.

The system should maintain a history of generated ideas so that future generation requests can exclude previously used topics or substantially similar ideas where practical.

17. Fresh Scraping

The user should be able to scrape competitors again after the initial collection.

- 🕒 Existing competitors can be selected for another scraping run.
- 🕒 The system should collect newly available updates.
- 🕒 Previously stored updates should be skipped.
- 🕒 New data should be added to the repository.
- 🕒 Scraping statistics should reflect the new run.

18. AI Providers

Students must use their own available free/developer AI API access for testing. Paid API usage is not required for this assignment.

At least two AI providers should be integrated. Suitable examples include:

- 🕒 Gemini
- 🕒 Grok
- 🕒 Another provider with an appropriate free/developer API

AI credentials must remain private and must not be shared with other students or exposed in the frontend, source repository, screenshots, or public documentation.

19. Web Interface

The complete system must have a simple, user-friendly web interface.

A non-technical user should be able to:

- 🕒 Create a project
- 🕒 Add the business profile
- 🕒 Add competitors
- 🕒 Add keywords
- 🕒 Start scraping
- 🕒 View scraping progress/status
- 🕒 Browse collected posts
- 🕒 View images and content
- 🕒 Run analysis

- 🕒 View trends
- 🕒 Generate a selected number of content ideas
- 🕒 Generate complete Google Maps updates
- 🕒 Review previously generated content

The interface must be responsive and usable on both PC and mobile.

20. Dashboard

The dashboard should provide a clear overview of project activity.

- 🕒 Number of competitors
- 🕒 Total posts collected
- 🕒 New posts from the latest scrape
- 🕒 Duplicate posts skipped
- 🕒 Failed scraping attempts
- 🕒 Last scraping activity
- 🕒 Top topics
- 🕒 Most frequently used keywords
- 🕒 Generated content count

21. Scraping Logs and Statistics

Every scraping run should produce persistent logs.

- 🕒 Competitor processed
- 🕒 Start time
- 🕒 End time
- 🕒 Posts found
- 🕒 New posts added
- 🕒 Duplicates skipped
- 🕒 Images downloaded
- 🕒 Failures
- 🕒 CAPTCHA/manual intervention status
- 🕒 Error information

22. Hosting Requirement

The completed web application must be hosted online and available for evaluation. The evaluator should be able to access the system without logging in.

- 🕒 Do not password-protect the evaluation interface.
- 🕒 Do not submit localhost as the only implementation.
- 🕒 The deployed application should remain available during evaluation.
- 🕒 The live environment must contain the demo dataset.
- 🕒 The live environment must expose the functionality needed to test the system.

23. Technology Requirements

The scraper/data-processing layer should be Python-based. The web application may use PHP or another suitable web technology.

Students are expected to build the system from scratch. AI tools, libraries, frameworks and documentation may be used during development.

The final system must demonstrate understanding of web development, Python automation, data storage, API integration, AI processing, data analysis and deployment.

24. Mandatory Demonstration

15. Create a project.
16. Add multiple competitors manually.
17. Add project keywords.
18. Show the scraping workflow.
19. Show the stored competitor updates.
20. Show images and post content.
21. Show duplicate detection by running another scrape.
22. Show AI analysis of the repository.
23. Show topic/trend analysis.
24. Generate a user-selected number of content ideas.
25. Generate a complete Google Maps update.
26. Show that previously generated ideas are not unnecessarily repeated.
27. Show the live deployed system.
28. Submit a video demonstrating the overall workflow.

25. Bonus Features

The following are optional enhancements.

- 🕒 Automatically discover competitor businesses from user-entered keywords such as 'salon in Kharghar' or 'salon in Bandra', then add relevant businesses to the scraping queue.
- 🕒 Automatically identify what the client's own Google Maps profile is missing compared with competitor activity.
- 🕒 Identify topics competitors are using that the client's profile has not covered.
- 🕒 Generate recommendations for improving the client's Google Maps update strategy.
- 🕒 Automatic model discovery and selection for AI providers.
- 🕒 Support additional AI providers.
- 🕒 Advanced trend visualizations and historical trend tracking.
- 🕒 Advanced similarity detection between competitor posts and generated content.
- 🕒 Automated scheduling of periodic scraping runs.
- 🕒 Reliable CAPTCHA reduction or permitted third-party CAPTCHA-solving integration. A demonstrated working solution earns the special bonus specified for this assignment.

26. Final Submission

Students must submit:

- 🕒 Live hosted application URL
- 🕒 Working demo project with pre-populated data
- 🕒 Working scraping demonstration
- 🕒 Video demonstrating the system
- 🕒 Sample competitor data collected by the system
- 🕒 AI analysis output
- 🕒 Generated content examples
- 🕒 List of AI providers used

- 🕒 Brief architecture explanation
- 🕒 Any bonus features implemented

27. Important Evaluation Principle

This is an examination of the complete system, not just a scraper. The strongest implementations will demonstrate a clear flow from competitor collection to persistent data repository, analysis, trend identification, content planning, and final Google Maps update generation.

The application should be reliable enough to recover from expected scraping and AI-provider failures. A temporary Google Maps restriction should not make the entire platform unusable because the stored repository and demo dataset must remain available.